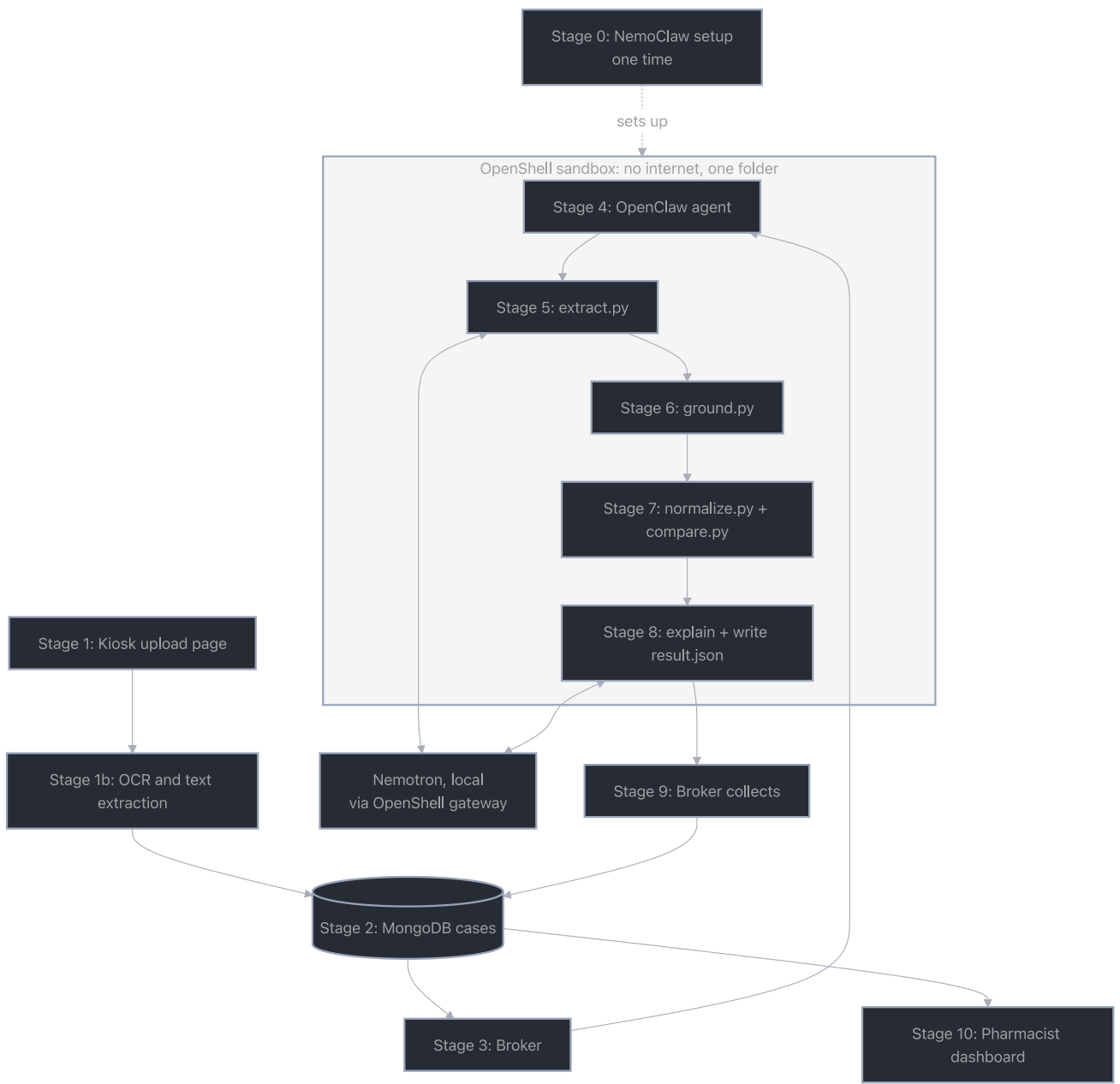


ScriptGuard MVP: architecture and data flow

Setup: one NemoClaw, one OpenShell sandbox, one OpenClaw agent, one local Nemotron model, all on one Dell GB10. Cases are processed one at a time, in token order.

Story: a discharged patient arrives at the hospital pharmacy, uploads their discharge summary and prescription at the check-in kiosk, and receives a token number. While they wait in line, ScriptGuard checks the two documents against each other. When their token is called, the pharmacist already sees every mismatch on the dashboard.

1. Architecture at a glance



Where each part runs

Location	Components
Host (outside sandbox)	Kiosk page, OCR, MongoDB, broker,

	dashboard
Inside OpenShell sandbox	OpenClaw agent and the Python tools it calls
Model, reached only through the OpenShell gateway	Nemotron

Two rules that hold everywhere

1. The broker's wake-up signal only says "something new arrived." It always claims the **oldest** pending case, so token order is kept even if several patients upload while one case is processing.
2. The agent never receives patient text in its message. It gets a case ID; the tools read the files.

2. Stage reference

Stage	Component	Runs on	Input (format)	Output (format)
0	NemoClaw setup	Host, one time	Policy + inference profile config	Running sandbox, OpenClaw agent, route to local Nemotron. Not part of the runtime loop
1	Kiosk upload page	Host, local network	HTTP multipart POST: name , mrn (strings), summary , prescription (PDF, scanned PDF, JPG or PNG)	Files saved to uploads/<token>/ , token shown to patient
1b	OCR and text extraction	Host, Python	PDF or image files	UTF-8 text (one line per text line) + per-page metadata (method, OCR confidence) + SHA-256 of each original file
2	MongoDB cases	Host	BSON document, <code>_id = token</code> , <code>status: "pending"</code> , both texts as strings	Insert event on the change stream
3	Broker	Host, Python	Insert event (wake-up signal only)	Claims oldest pending case; writes <code>summary.txt</code> , <code>prescription.txt</code> , <code>meta.json</code> into the sandbox case folder; sends agent the message <code>Process case <token></code>
4	OpenClaw agent	Sandbox	Message string with case ID only, fresh session per case	Runs the ScriptGuard skill: stages 5 to 8 in order

5	<code>extract.py</code>	Sandbox + Nemotron	Two .txt files, turned into numbered lines	Two JSON objects: list of medicines with <code>line</code> , <code>span</code> , <code>raw</code> fields
6	<code>ground.py</code>	Sandbox	Extraction JSON + source lines	List of errors (empty = grounded). On errors, agent retries extraction once, then marks REVIEW
7	<code>normalize.py</code> + <code>compare.py</code>	Sandbox	Grounded extraction JSON + <code>brands.json</code> , <code>frequencies.json</code>	List of flags: <code>severity</code> , <code>drug</code> , <code>type</code> , both source lines
8	Explain + write result	Sandbox + Nemotron	Flags list	<code>result.json</code> : status, flags with one-sentence explanations
9	Broker collects	Host	<code>result.json</code>	MongoDB update (<code>status</code> : "ready" or "review", flags); deletes the sandbox case folder; claims next pending case
10	Pharmacist dashboard	Host	MongoDB query: cases sorted by token	On close: <code>status</code> : "closed", raw texts and uploaded files removed, audit fields kept

3. Flow example: token T-042

Stage 0. NemoClaw setup (once, before the pharmacy opens)

NemoClaw creates the OpenShell sandbox with this policy:

- No internet access.
- Filesystem access limited to the ScriptGuard folder (`case/` for input, results written into the same case folder).
- Model calls routed through the OpenShell gateway to the **local** Nemotron model.

It installs the OpenClaw agent with the ScriptGuard skill. Nothing patient-related happens here.

Stage 1. Kiosk upload

Mary enters her details and uploads two files:

Field	Value	Type
<code>name</code>	Mary Thomas	string
<code>mrn</code>	48213	string

summary	discharge_summary.pdf	scanned PDF, 1 page, image only
prescription	prescription.pdf	digital PDF, 1 page, has a text layer

The kiosk saves both originals to uploads/T-042/ and shows: **Your token is T-042.**

Stage 1b. OCR and text extraction

For each page:

1. If the page's text layer contains a medicine-dose pattern (like "500 mg"), use the text layer. It's exact and fast.
2. Otherwise (a scanned page, or a page whose medicines are inside an image), render the page at 300 dpi and run Tesseract OCR on it.
3. JPG and PNG uploads go straight to OCR.

Then clean the text (trim whitespace, drop empty lines) **before** storing it. Line numbers used later for grounding are counted on this cleaned text, so the stored text must be exactly what gets numbered.

Output for T-042

summary (OCR, confidence 91.4)	prescription (text layer)
Patient: Mary Thomas, MRN 48213	Patient: Mary Thomas, MRN 48213
Continue metformin 500 mg BID.	Metformin 1000 mg BID
Continue sertraline 50 mg OD.	Paracetamol 650 mg TDS
Start Dolo 650 TDS for pain.	

```
"source_files": [
  {"doc": "summary", "filename": "discharge_summary.pdf", "sha256": "9f2c41...",
   "pages": [{"page": 1, "method": "ocr", "confidence": 91.4}]},
  {"doc": "prescription", "filename": "prescription.pdf", "sha256": "b7e0a3...",
   "pages": [{"page": 1, "method": "text_layer"}]}
]
```

OCR code (offline: PyMuPDF + Tesseract)

```
import io, re, hashlib, fitz, pytesseract
from PIL import Image

DOSE = re.compile(r"\d+(\.\d+)?\s*(mg|mcg|g|ml|units?|iu)\b", re.I)

def ocr_image(img):
    data = pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT)
    confs = [float(c) for c in data["conf"] if float(c) >= 0]
    return pytesseract.image_to_string(img), round(sum(confs) / len(confs), 1) if confs else 0

def file_to_text(path):
    pages, meta = [], []
    if path.lower().endswith((".jpg", ".jpeg", ".png")):
        text, conf = ocr_image(Image.open(path))
        pages, meta = [text], [{"page": 1, "method": "ocr", "confidence": conf}]
    else:
        for p in fitz.open(path):
            layer = p.get_text("text")
```

```

    layer = page_text_text
    if DOSE.search(layer):
        pages.append(layer); meta.append({"page": p.number + 1, "method": "text_l
    else:
        img = Image.open(io.BytesIO(p.get_pixmap(dpi=300).tobytes("png")))
        text, conf = ocr_image(img)
        pages.append(text); meta.append({"page": p.number + 1, "method": "ocr", "
    lines = [l.strip() for l in "\n".join(pages).splitlines() if l.strip()]
    sha = hashlib.sha256(open(path, "rb").read()).hexdigest()
    return "\n".join(lines), meta, sha

```

Install today, while you have internet: tesseract-ocr (system package), pytesseract, pymupdf, pillow. The venue Wi-Fi may not cooperate.

Known OCR behavior: OCR sometimes misreads characters (for example "500 mg" instead of "500 mg"). That doesn't break anything: the misread value is really in the stored text, so it passes grounding, then fails to match the other document and shows up as a HIGH flag with the source line. The pharmacist sees the odd text and checks the original.

Stage 2. MongoDB insert

```

{
  "_id": "T-042",
  "status": "pending",
  "created_at": "2026-10-04T14:02:11Z",
  "patient": {"name": "Mary Thomas", "mrn": "48213"},
  "summary_text": "Patient: Mary Thomas, MRN 48213\nContinue metformin 500 mg BID.\nConti
  "prescription_text": "Patient: Mary Thomas, MRN 48213\nMetformin 1000 mg BID\nParacetam
  "source_files": [ "...as in stage 1b..." ]
}

```

Stage 3. Broker

The insert event arrives. The broker claims the oldest pending case in one atomic step, so status becomes "processing". It writes this folder into the sandbox:

```

case/T-042/summary.txt
case/T-042/prescription.txt
case/T-042/meta.json      {"case_id": "T-042", "mrn": "48213"}

```

Then it sends the agent the message `Process case T-042`.

Stage 4. OpenClaw agent

In a fresh session, the agent follows the ScriptGuard skill and calls the tools in order on `case/T-042/`.

Stage 5. Extraction

`extract.py` sends Nemotron one request per document, through the gateway's OpenAI-style API:

```

{"model": "<your-nemotron>", "temperature": 0,
 "messages": [{"role": "system", "content": "<extraction rules>"},
               {"role": "user", "content": "case_id: T-042\n1 Patient: Mary Thomas, MRN 48

```

Nemotron returns a text reply containing JSON, which the tool parses:

```
{"case_id": "T-042", "doc": "prescription", "meds": [
  {"line": 2, "span": "Metformin 1000 mg BID", "name_raw": "Metformin",
   "dose_raw": "1000", "unit_raw": "mg", "freq_raw": "BID", "status": "continue"},
  {"line": 3, "span": "Paracetamol 650 mg TDS", "name_raw": "Paracetamol",
   "dose_raw": "650", "unit_raw": "mg", "freq_raw": "TDS", "status": "continue"}]}
```

The summary gets the same treatment, producing three medicines (lines 2 to 4).

Stage 6. Grounding

Every span is found on its claimed line, and every raw value sits inside its span. The returned `case_id` matches. Result: [] (no errors).

If this had failed, the agent would re-run extraction once with the error included, then mark the case REVIEW.

Stage 7. Normalize and compare

```
[
  {"severity": "HIGH", "drug": "metformin", "type": "dose_mismatch",
   "summary": "Continue metformin 500 mg BID. (line 2)",
   "prescription": "Metformin 1000 mg BID (line 2)"},
  {"severity": "HIGH", "drug": "sertraline", "type": "missing_from_prescription",
   "summary": "Continue sertraline 50 mg OD. (line 3)", "prescription": null},
  {"severity": "LOW", "drug": "paracetamol", "type": "name_differs_confirm_brand_intent",
   "summary": "Start Dolo 650 TDS for pain. (line 4)",
   "prescription": "Paracetamol 650 mg TDS (line 3)"}
]
```

Stage 8. Explain and write result

The agent asks Nemotron for one sentence per flag and writes `case/T-042/result.json`:

```
{"case_id": "T-042", "status": "ready", "checked": 3,
 "flags": [
  {"severity": "HIGH", "drug": "metformin", "type": "dose_mismatch",
   "explanation": "The prescription doubles the metformin dose written in the summary."},
  {"severity": "HIGH", "drug": "sertraline", "type": "missing_from_prescription",
   "explanation": "Sertraline is continued in the summary but not on the prescription."},
  {"severity": "LOW", "drug": "paracetamol", "type": "name_differs_confirm_brand_intent",
   "explanation": "Summary names Dolo 650, prescription names generic paracetamol; confir
```

Stage 9. Broker collects

The broker copies the result into MongoDB (`status: "ready"` , plus `flags`), deletes `case/T-042/` from the sandbox, and claims the next pending case, T-043.

Stage 10. Pharmacist dashboard

When T-042 is called, the pharmacist sees two HIGH flags and one LOW flag, with the source lines side by side. After resolving them, they click **Close case**:

```
{" id": "T-042". "status": "closed". "closed at": "2026-10-04T14:19:40Z".
```

```
"flags": [ "...kept..." ],
"pharmacist_decision": "Metformin corrected to 500 mg; sertraline added; Dolo confirmed"
"doc_fingerprints": ["sha256:9f2c41...", "sha256:b7e0a3..."]}
```

The raw texts (`summary_text` , `prescription_text`) are removed with `$unset` , the originals in `uploads/T-042/` are deleted, and the audit record stays.

4. The broker

```
from datetime import datetime, timezone
from pymongo import MongoClient, ReturnDocument

db = MongoClient("mongodb://localhost:27017/?replicaSet=rs0").scriptguard

def claim_next():
    return db.cases.find_one_and_update(
        {"status": "pending"},
        {"$set": {"status": "processing", "claimed_at": datetime.now(timezone.utc)}},
        sort=[("created_at", 1)], return_document=ReturnDocument.AFTER)

def drain_queue():
    while (case := claim_next()):
        stage_into_sandbox(case)          # write the 3 files
        send_to_agent(f"Process case {case['_id']}")
        result = wait_for_result(case["_id"]) # poll for result.json
        db.cases.update_one({"_id": case["_id"]}, {"$set": result})
        remove_from_sandbox(case["_id"])

drain_queue()                            # catch anything already waiting
with db.cases.watch([{"$match": {"operationType": "insert"}}]) as stream:
    for _ in stream:
        drain_queue()
```

Fill in `stage_into_sandbox` , `send_to_agent` , `wait_for_result` , and `remove_from_sandbox` once you've confirmed (in the first 30 minutes) how files get into the sandbox and how to message the agent from the host.

Setup catch: MongoDB change streams only work on a replica set. On one machine, start MongoDB with `--replSet rs0` and run `rs.initiate()` once. If that causes trouble, replace the `watch` loop with `drain_queue()` every 2 seconds. The demo looks the same.

5. End-of-day audit report

The dashboard's audit view is one MongoDB aggregation over today's closed cases:

```
start = datetime.now(timezone.utc).replace(hour=0, minute=0, second=0, microsecond=0)
report = db.cases.aggregate([
    {"$match": {"status": "closed", "closed_at": {"$gte": start}}},
    {"$project": {"flags": 1, "high": {"$size": {"$filter": {"input": "$flags", "cond": {
    {"$group": {"_id": None, "cases": {"$sum": 1},
        "with_flags": {"$sum": {"$cond": [{"$gt": [{"$size": "$flags"}, 0]}, 1, 0
```

```
"high_flags": {"$sum": "$high"}}}
```

```
] )
```

The report shows cases checked, cases with mismatches, HIGH and LOW flag counts, cases sent to review, and each closed case's token, flags, and pharmacist decision. It contains no document text, since that was removed when each case closed.

